VLA-Corrector: Lightweight Detect-and-Correct Inference for Adaptive Action Horizon

Yi Pan, Miao Pan, Qi Lu, Jiaming Huang, Man Zhang, Siteng Huang, Xin Li, Jie Zhang, Yongliang Shen, Xuhong Zhang, Wenqi Zhang

ABSTRACT

Vision-Language-Action (VLA) foundation models have recently achieved strong progress in embodied intelligence. To reduce policy-call frequency while preserving temporal coherence, most generative policies adopt an action chunk mechanism, executing multiple future actions in an open-loop manner under a fixed action horizon. However, this "predict-then-blindly-execute" paradigm sacrifices closed-loop reactivity: in contact-rich physical interactions, even small local perturbations can rapidly amplify within the open-loop blind spot, leading to compounding errors and ultimately task failure. To address this limitation, we propose VLA-Corrector, a lightweight corrective inference framework for action-chunked VLA policies. Without modifying the backbone policy weights, VLA-Corrector introduces a lightweight Latent-space Vision Monitor (LVM) that continuously compares predicted and actual visual feature evolution, enabling online detection of visual dynamics deviations. Once persistent deviation is detected, the system triggers a truncation event, discards the remaining stale actions, and invokes corrective replanning via Online Gradient Guidance (OGG). The detect-and-correct mechanism of VLA-Corrector naturally induces an event-triggered adaptive action horizon: it preserves long-horizon execution when the current chunk remains reliable, and invokes short-horizon corrective replanning when execution begins to drift. In doing so, VLA-Corrector mitigates the trade-off imposed by static horizons between execution robustness and policy-call frequency. It can be integrated into different VLA models without further retraining the VLA backbone, interrupting compounding errors while preserving much of the efficiency benefit of action chunking and substantially improving robustness in long-horizon, contact-rich robotic manipulation tasks.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.
Provenance. Ingested by llmXive on 2026-07-07 — scraped from arXiv; via llmXive dashboard,

GitHub issue #522; submitted by github-actions[bot].

Source. https://arxiv.org/abs/2607.01804

About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at https://github.com/ContextLab/llmXive.

Automated review. See the accompanying peer-review-llmxive.pdf for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: PROJ-999-llmxive-follow-up-extending-vla-correcto.

OmniAI Group of ZJU ACES Lab
Zhejiang University

VLA-Corrector: Lightweight Detect-and-Correct Inference for Adaptive Action Horizon

Yi Pan[1], **Miao Pan**[1], **Qi Lu**[1], **Jiaming Huang**[1], **Man Zhang**[1], **Siteng Huang**[2], **Xin Li**[2], **Jie Zhang**[1], **Yongliang Shen**[1], **Xuhong Zhang**[1], **Wenqi Zhang**[1]

[1]Zhejiang University [2]Alibaba DAMO Academy

Vision-Language-Action (VLA) foundation models have recently achieved strong progress in embodied intelligence. To reduce policy-call frequency while preserving temporal coherence, most generative policies adopt an **action chunk** mechanism, executing multiple future actions in an open-loop manner under a fixed **action horizon**. However, this "predict-then-blindly-execute" paradigm sacrifices closed-loop reactivity: in contact-rich physical interactions, even small local perturbations can rapidly amplify within the open-loop blind spot, leading to compounding errors and ultimately task failure. To address this limitation, we propose **VLA-Corrector**, a lightweight corrective inference framework for action-chunked VLA policies. Without modifying the backbone policy weights, VLA-Corrector introduces a lightweight **Latent-space Vision Monitor** (LVM) that continuously compares predicted and actual visual feature evolution, enabling online detection of visual dynamics deviations. Once persistent deviation is detected, the system triggers a truncation event, discards the remaining stale actions, and invokes corrective replanning via **Online Gradient Guidance** (OGG). The detect-and-correct mechanism of VLA-Corrector naturally induces an **event-triggered adaptive action horizon**: it preserves long-horizon execution when the current chunk remains reliable, and invokes short-horizon corrective replanning when execution begins to drift. In doing so, VLA-Corrector mitigates the trade-off imposed by static horizons between execution robustness and policy-call frequency. It can be integrated into different VLA models without further retraining the VLA backbone, interrupting compounding errors while preserving much of the efficiency benefit of action chunking and substantially improving robustness in long-horizon, contact-rich robotic manipulation tasks. Our code is available at https://github.com/ZJU-OmniAI/vla-corrector.

Correspondence: Wenqi Zhang
Email: Yi Pan panyi0304@gmail.com; Wenqi Zhang zhangwenqi@zju.edu.cn
Date: June 2026

1 Introduction

Vision-Language-Action (VLA) foundation models have emerged as a promising path toward general-purpose robot control, unifying perception, language, and action generation within a single framework Zitkovich et al. (2023); O'Neill et al. (2024); Kim et al. (2024); Black et al. (2024); Li et al. (2023); Bjorck et al. (2025); Intelligence et al. (2025); Ma et al. (2024); Wang et al. (2026a); Cen et al. (2025); Jiang et al. (2025b). Modern VLA policies often rely on generative action models, such as diffusion models and flow matching, to capture the high-dimensional and multi-modal nature of continuous robot actions Black et al. (2024); Chi et al. (2025); Jiang et al. (2025a); Wang et al. (2024); Lipman et al. (2022). However, the per-step latency of generative models precludes closed-loop replanning, creating a fundamental tension between action expressiveness and high-frequency feedback control.

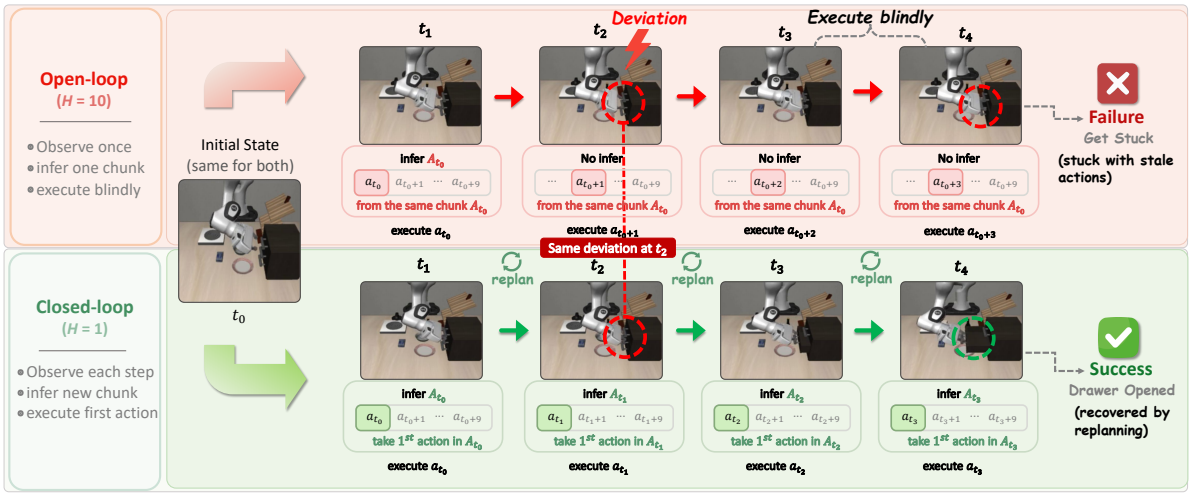


Figure 1 Open-loop vs. Closed-loop execution. The top row ($H = 10$) illustrates how blindly executing long action chunks leads to compounding errors, causing the robot to get stuck during a drawer-opening task. In contrast, the bottom row ($H = 1$) demonstrates strict closed-loop execution, which maintains environmental reactivity and yields a smooth, successful manipulation.

A common engineering compromise is to use an **action chunk** Zhao et al. (2023a): in a single forward pass, the policy predicts a sequence of future actions, while the controller only executes the first H steps as the **action horizon** Jing et al. (2025); Liu et al. (2024); Sendai et al. (2025); Chi et al. (2025); Khan and Tanimura (2025); So et al. (2025). This design amortizes policy-call frequency and improves temporal smoothness, but it also creates an **open-loop blind spot** Wang et al. (2026b); Xu et al. (2026) that introduces two compounding risks. **First, the policy lacks real-time reactivity**. Fresh observations arrive at every control step, yet the system ignores them until the horizon ends, leaving it unable to respond to unexpected slippage, collision, or pose drift as they occur. **Second, errors can accumulate beyond the point that replanning can recover from during the blind spot**. If deviations go uncorrected long enough, the robot may drift into an out-of-distribution state that the policy has rarely encountered during training, at which point even the next replanning call cannot steer execution back to the intended trajectory Wang et al. (2026b); Liu et al. (2024); Sendai et al. (2025); Wu et al. (2026a); Xu et al., and the task ultimately fails. Crucially, both risks worsen as the horizon grows longer. As illustrated in Fig. 1, under a long horizon ($H = 10$), the robot keeps executing stale actions after a deviation and eventually gets stuck; in contrast, with $H = 1$, the policy replans at every step and avoids the same error accumulation.

Yet $H = 1$ is not a practical solution: it requires a full VLA inference at every control step, negating the efficiency gains that action chunking was designed to provide. The contrast between these two extremes thus exposes a fundamental trade-off: smaller horizons improve closed-loop responsiveness but multiply policy-call frequency, while larger horizons amortize computation but widen the blind spot where errors silently accumulate.

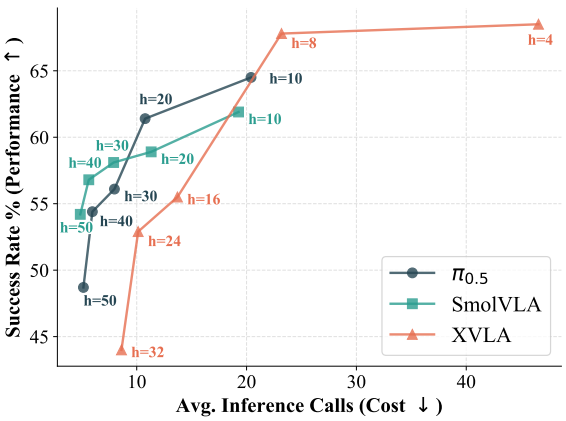


Figure 2 Performance–efficiency trade-off across fixed action horizons. Smaller horizon achieves higher success rates, while larger horizon preserves the chunking efficiency.

To quantify this trade-off, we conduct systematic experiments comparing task success rate and policy frequency across a range of action horizons on three VLA backbones. As shown in Fig. 2, larger horizons substantially reduce policy calls but also lead to lower success rates, and this trend is consistently observed across all three

backbones. For $\pi_{0.5}$ Intelligence et al. (2025), for example, increasing the horizon reduces policy calls by about 4×, but lowers success from about 64% to below 49%; similar trends hold for SmolVLA Shukor et al. (2025) and X-VLA Zheng et al. (2025). Furthermore, since the best horizon depends on task difficulty, environmental dynamics, and sim-to-real mismatch, it is difficult to predefine a single static horizon that remains optimal across scenarios. This suggests that the key is not to choose a better fixed horizon, but to decide when the current chunk should stop being trusted.

The above analysis raises two key questions: (1) How to **detect execution deviations in time** and **terminate stale actions** before errors compound beyond recovery; (2) How to **correct the trajectory after truncation**—since naive replanning alone is often insufficient: the VLA may re-generate actions that still fail to escape the deviated state, leaving the robot trapped again.

Motivated by these insights, we propose **VLA-Corrector**, a lightweight corrective inference framework that addresses both questions. For the first, VLA-Corrector introduces a **Latent-space Vision Monitor** (LVM) that continuously compares predicted and actual visual evolution during open-loop execution; once persistent deviation is detected, it triggers an **interrupt event** that immediately truncates the remaining stale actions and invokes a new action generation. For the second, rather than relying on naive replanning, VLA-Corrector activates **Online Gradient Guidance** (OGG), which exploits the discrepancy between predicted and observed visual dynamics to inject a corrective gradient during new action generation—actively steering the robot back toward the intended trajectory, instead of simply hoping that re-inference will naturally recover. Together, LVM-triggered truncation and OGG-guided re-inference transform a fixed action horizon into an **adaptive action horizon** with **self-correcting re-inference**, preserving long-horizon efficiency and short-horizon responsiveness.

These two mechanisms jointly improve success-per-call efficiency: the robot avoids wasting long chunks on stale actions, so each policy query contributes more effectively to task completion. For $\pi_{0.5}$ at horizon 50, success rises from 48.7% to 58.7% while average policy calls drop from 5.15 to 4.98—a +24.6% success-per-call efficiency gain. Gains are consistent across all three backbones and grow larger at longer horizons, where the open-loop blind spot is widest. Beyond robustness, VLA-Corrector also improves sample efficiency: on LIBERO, a few-shot fine-tuned model augmented with VLA-Corrector reaches 97.8% average success, *surpassing* the fully fine-tuned baseline (96.9%).

Our contributions are summarized as follows:

- We systematically quantify the performance–efficiency trade-off induced by fixed action horizon in action-chunked VLA policies, showing that the open-loop blind spot consistently affects robustness across different VLA backbones.
- We propose VLA-Corrector, a lightweight inference-time framework that augments a frozen VLA backbone with latent-space monitoring, event-triggered truncation, and guidance toward recovery-oriented inference.
- We validate VLA-Corrector across simulation and real-world tasks, showing that it achieves stronger robustness, higher success-per-call efficiency, and adaptive correction behavior.

2 Preliminaries

VLA Policies with Action Chunks. In embodied continuous control, the current visual observation o_t is first encoded into a latent representation $Z_t^{\text{real}} = \mathcal{E}(o_t)$ by a visual encoder $\mathcal{E}$. To balance policy-call frequency and action smoothness, modern generative VLA policies parameterized by θ commonly predict an **action chunk** in a single inference call,

$$A_t = [a_t, a_{t+1}, \ldots, a_{t+C-1}] \sim \pi_\theta(\cdot \mid Z_t^{\text{real}}, l), \tag{1}$$

where a_t denotes a single control action and l is the language instruction. During deployment, only the first H actions, with $H \leq C$, are executed sequentially, and we refer to this execution window as the *action horizon*. The corresponding execution queue is $Q_t = [a_t, a_{t+1}, \ldots, a_{t+H-1}]$, during which the controller does not query the VLA policy again.

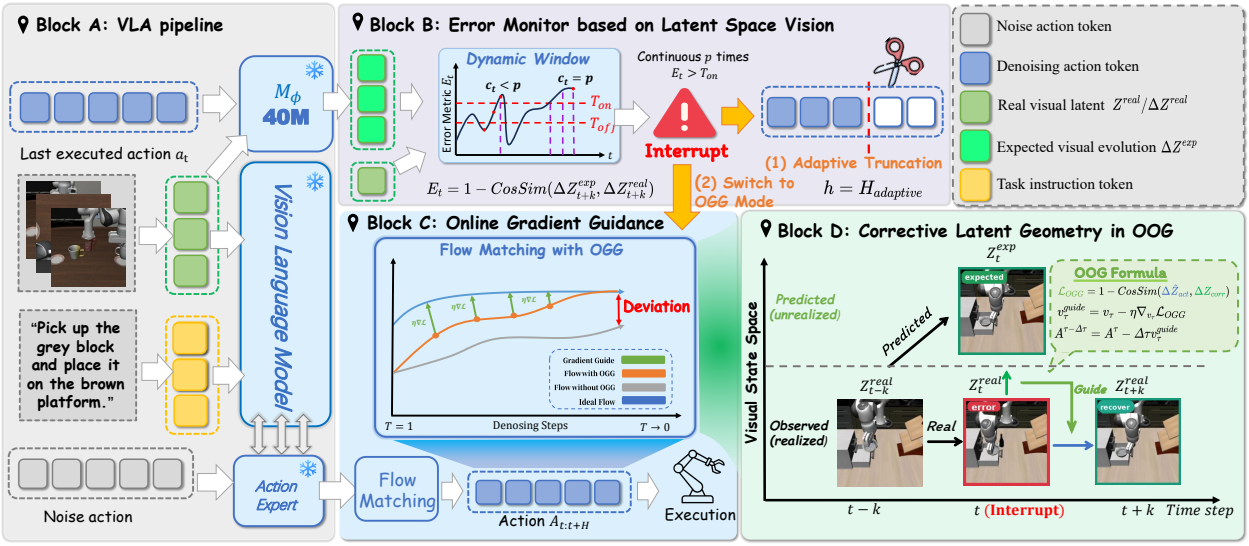


Figure 3 Overview of VLA-Corrector. Starting from a standard chunked VLA pipeline (**A**), we add a **Latent-space Vision Monitor** (LVM) that detects persistent execution drift and triggers an interrupt event (**B**). The event truncates stale actions and switches the next replan from normal flow matching to OGG-guided flow matching (**C**). OGG uses the expected and observed latent evolution to guide the replan back toward a recoverable trajectory (**D**).

3 VLA-Corrector: A Lightweight Corrective Inference Framework

VLA-Corrector addresses the open-loop blind spot of action-chunked VLA policies. During the action horizon, fresh observations are available, but the policy does not use them until the horizon ends. We therefore decouple *action generation* from *execution monitoring*: the VLA backbone generates action chunks, while VLA-Corrector monitors whether execution remains on track and intervenes only when drift emerges. We organize the method as follows: **(1) Corrector Training** describes the external latent dynamics corrector; **(2) LVM Detection** introduces online visual deviation detection; **(3) Event-Triggered Truncation** explains how persistent deviations induce adaptive horizons; and **(4) OGG Correction** presents corrective replanning after truncation.

3.1 Training the External Latent Dynamics Corrector

VLA-Corrector is trained after a VLA policy has already been obtained. We first fine-tune the VLA backbone on the benchmark training set, then freeze the VLA and use its visual encoder $\mathcal{E}$ to extract visual latents from demonstration trajectories. Given a transition (o_t, a_t, o_{t+k}), we compute

$$Z_t^{\text{real}} = \mathcal{E}(o_t), \qquad Z_{t+k}^{\text{real}} = \mathcal{E}(o_{t+k}), \qquad \Delta Z_{t+k}^{*} = Z_{t+k}^{\text{real}} - Z_t^{\text{real}}, \tag{2}$$

where ΔZ_{t+k}^{*} is the target short-horizon visual latent evolution induced by the executed action. We then train a lightweight external dynamics corrector M_ϕ to predict this residual evolution from the current latent state and action:

$$\Delta \hat{Z}_{t+k} = M_\phi(Z_t^{\text{real}}, a_t). \tag{3}$$

Rather than predicting the absolute future latent state, M_ϕ instead predicts a residual visual latent evolution, which actively suppresses static scene content and encourages the model to focus on task-relevant dynamics. The training objective combines magnitude matching and directional consistency:

$$\mathcal{L}_{\text{corr}} = \left\| \Delta \hat{Z}_{t+k} - \Delta Z_{t+k}^{*} \right\|_2^2 + \beta \left[1 - \text{CosSim}\left(\Delta \hat{Z}_{t+k}, \Delta Z_{t+k}^{*} \right) \right], \tag{4}$$

where β balances residual accuracy and directional alignment. The trainable component is therefore not the VLA policy itself, but a lightweight latent dynamics module trained separately on top of frozen VLA features. This modular design allows the corrector to be retrained or replaced for each benchmark without re-optimizing

the expensive VLA backbone. The corrector is trained on benchmark demonstration trajectories because they are easy to obtain and broadly reflect successful task progress. Its goal is not to model all possible futures as a full world model would do, but to learn whether local latent dynamics remain consistent with on-track execution. Demonstrations are suitable for this purpose: although they may contain small teleoperation jitter or local imperfections, they still capture behavior that keeps the task progressing correctly. Since this is a local and low-dimensional prediction task, a lightweight MLP is sufficient in practice; in our experiments, a 40M corrector already provides an effective monitoring and corrective signal, which can be trained at a very low cost.

3.2 Latent Visual Dynamics for Online Anomaly Detection

After training M_ϕ, we use it online to monitor whether the current action chunk remains reliable. During execution, **Latent-space Vision Monitor** (LVM) compares the *expected* latent visual evolution predicted by M_ϕ with the *actual* evolution observed from fresh camera input, detecting deviations before they accumulate into failure.

Expected residual dynamics. At control step t, LVM uses the executed action a_t and current latent state Z_t^{real} to predict the expected short-horizon latent residual, $\Delta Z_{t+k}^{\text{exp}} = M_\phi(Z_t^{\text{real}}, a_t)$, where k is the prediction interval.

Online inconsistency score. During rollout, the latest observation remains available even though the policy is not re-queried. We encode the future observation and compute the actual residual $\Delta Z_{t+k}^{\text{real}} = Z_{t+k}^{\text{real}} - Z_t^{\text{real}}$. LVM then measures the mismatch between expected and actual latent evolution:

$$E_t = 1 - \text{CosSim}\left(\Delta Z_{t+k}^{\text{exp}}, \Delta Z_{t+k}^{\text{real}}\right), \qquad \text{CosSim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top \mathbf{v} / (\|\mathbf{u}\| \, \|\mathbf{v}\|). \tag{5}$$

A larger E_t indicates stronger visual dynamics mismatch, providing a continuous signal for later event-triggered truncation.

3.3 Event-Triggered Truncation under Robust Online Monitoring

Given the continuous inconsistency score E_t, we need to decide when a deviation is persistent enough to justify intervention. Directly thresholding E_t can be unstable, since transient visual outliers may cause false triggers. We therefore use a robust event-triggered rule based on dynamic thresholds and persistence checking. Specifically, we maintain a sliding window of recent scores, $\mathbf{E}_W = \{E_{t-w+1}, \ldots, E_t\}$, and compute its median M_e and median absolute deviation,

$$\text{MAD} = \text{median}(|E_i - M_e|), \qquad E_i \in \mathbf{E}_W. \tag{6}$$

We then define two adaptive thresholds,

$$T_{\text{on}} = M_e + \lambda_{\text{on}}\text{MAD}, \qquad T_{\text{off}} = M_e + \lambda_{\text{off}}\text{MAD}, \qquad \lambda_{\text{on}} > \lambda_{\text{off}}, \tag{7}$$

where T_{on} confirms persistent deviation and T_{off} provides hysteresis for recovery. An interrupt event is triggered only when $E_t > T_{\text{on}}$ holds for p consecutive steps; isolated spikes are ignored.

Once an interrupt is triggered, VLA-Corrector discards the remaining actions in the current queue and queries the policy again under corrective mode. If the controller has executed h actions from the original queue, the realized horizon becomes $H_{\text{adaptive}} = h < H$. Thus, the fixed action horizon is shortened only when sustained visual drift indicates that the current chunk has become stale. The full thresholding state machine is provided in Appendix B.1.

3.4 Online Gradient Guidance for Corrective Inference

Truncation stops stale actions, but recovery still depends on the next replan. After an interrupt event, VLA-Corrector applies **Online Gradient Guidance** (OGG) Park et al. (2025) only to the single policy call immediately following the interrupt event, guiding this recovery replan toward a corrective latent direction.

Table 1 Cross-architecture generalization on MetaWorld. Success rate (%, ↑) across difficulty splits. Green arrows show absolute improvements over the corresponding baseline.

Backbone	Method	Easy↑	Medium↑	Hard↑	Very Hard↑	Avg.↑
$\pi_{0.5}$	Baseline	70.5	45.0	38.3	41.0	48.70
	+ **VLA-Corrector**	**83.2** ↑ 12.7	**61.7** ↑ 16.7	**47.5** ↑ 9.2	**65.0** ↑ 24.0	**64.35** ↑ 15.65
SmolVLA	Baseline	81.3	53.6	51.7	61.0	61.90
	+ **VLA-Corrector**	**83.4** ↑ 2.1	**56.0** ↑ 2.4	**64.2** ↑ 12.5	**63.0** ↑ 2.0	**66.65** ↑ 4.75
X-VLA	Baseline	72.5	46.4	48.3	55.0	55.55
	+ **VLA-Corrector**	**74.4** ↑ 1.9	**50.0** ↑ 3.6	**50.0** ↑ 1.7	**64.0** ↑ 9.0	**59.60** ↑ 4.05

Table 2 Sample efficiency on LIBERO. Success rate (%, ↑). Green arrows compare VLA-Corrector against the few-shot fine-tuned baseline.

Model	Object↑	Spatial↑	Goal↑	Long↑	Avg.↑
$\pi_{0.5}$ (Full Fine-tuned)	99.4	98.2	97.8	92.4	96.95
$\pi_{0.5}$ (Few-shot Fine-tuned)	97.8	95.4	96.2	86.6	94.00
$\pi_{0.5}$ (Few-shot) + **VLA-Corrector**	**99.8** ↑ 2.0	**100.0** ↑ 4.6	**98.0** ↑ 1.8	**93.4** ↑ 6.8	**97.80** ↑ 3.80

Predicted action effect. At denoising step τ of flow matching, let A^τ be the noisy action chunk. The VLA predicts a velocity field $v_\tau = \pi_\theta(A^\tau, Z_t^{\text{real}}, \tau)$, from which we estimate the clean chunk $\hat{A}_0 = A^\tau - \tau v_\tau$ and take its first action $\hat{a}_t = \hat{A}_0[0]$. The corrector then predicts the latent effect of this candidate action:

$$\Delta\hat{Z}_{\text{act}} = M_\phi(Z_t^{\text{real}}, \hat{a}_t). \tag{8}$$

Corrective target. Let $t-k$ be the last stable step before an interrupt event. We define the expected residual $\Delta Z_{\text{exp}} = \Delta Z_t^{\text{exp}} = M_\phi(Z_{t-k}^{\text{real}}, a_{t-k})$, which is predicted by M_ϕ from step $t-k$. The accumulated deviation is $\Delta Z_{\text{dev}} = \Delta Z_t^* = Z_t^{\text{real}} - Z_{t-k}^{\text{real}}$. The corrective latent direction is

$$\Delta Z_{\text{corr}} = \Delta Z_{\text{exp}} - \Delta Z_{\text{dev}}. \tag{9}$$

This direction preserves the intended local dynamics while compensating for the drift accumulated during open-loop execution.

Guided velocity update. OGG aligns the predicted action effect with ΔZ_{corr} through

$$\mathcal{L}_{\text{OGG}} = 1 - \text{CosSim}(\Delta\hat{Z}_{\text{act}}, \Delta Z_{\text{corr}}), \tag{10}$$

and injects the resulting gradient into the flow-matching velocity:

$$v_\tau^{\text{guide}} = v_\tau - \eta\nabla_{v_\tau}\mathcal{L}_{\text{OGG}}, \qquad A^{\tau-\Delta\tau} = A^\tau - \Delta\tau v_\tau^{\text{guide}}. \tag{11}$$

Here, η controls the guidance strength. Since OGG modifies the velocity field rather than directly perturbing action coordinates, it remains compatible with the original flow-matching process and yields smoother corrective replanning.

4 Experiments

We evaluate VLA-Corrector on MetaWorld Yu et al. (2020), LIBERO Liu et al. (2023), and a real AgileX PiPER robot, using $\pi_{0.5}$ Intelligence et al. (2025) as the main backbone and SmolVLA Shukor et al. (2025)/X-VLA Zheng et al. (2025) for cross-architecture evaluation. The experiments cover four aspects: **(1) Performance and policy-call efficiency**, evaluating success rate and success-per-call across benchmarks and horizons; **(2) Mechanism analysis**, examining LVM detection, truncation timing, and OGG recovery; **(3) Real-world transfer**, testing the method on physical robot manipulation; and **(4) Ablations**, analyzing component necessity and sensitivity. Detailed protocols, metrics, and compute resources are provided in Appendix C.1.

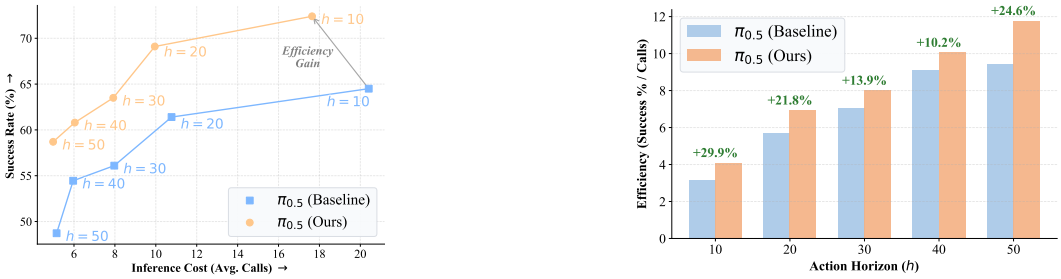


Figure 4 Performance–efficiency analysis on $\pi_{0.5}$. Left: performance–efficiency trade-off across action horizons. **Right**: success-per-call efficiency. VLA-Corrector improves success rate across action horizons and yields consistent efficiency gains.

Table 3 Data efficiency of corrector training on MetaWorld. Success rate (%, ↑). We compare the open-loop baseline at horizon 50 with VLA-Corrector trained using different fractions of demonstration trajectories. Green/red arrows indicate absolute change in average success over the baseline.

Method / Ratio	Easy↑	Medium↑	Hard↑	Very Hard↑	Avg.↑
Baseline	70.54	45.00	38.33	41.00	48.72
VLA-Corrector ($r = 0.2$)	71.07	49.55	36.67	36.00	48.32 ↓ 0.40
VLA-Corrector ($r = 0.4$)	70.36	45.91	38.33	42.00	49.15 ↑ 0.43
VLA-Corrector ($r = 0.6$)	70.89	52.73	39.17	**46.00**	52.20 ↑ 3.48
VLA-Corrector ($r = 0.8$)	71.07	53.64	38.33	**46.00**	52.26 ↑ 3.54
VLA-Corrector ($r = 1.0$)	**73.21**	**55.91**	**44.17**	44.00	**54.32** ↑ 5.60

4.1 Main Results

Cross-Architecture Evaluation on MetaWorld. VLA-Corrector consistently improves all three VLA backbones on MetaWorld, with larger gains on harder tasks. As shown in Table 1, the average success rate improves by 15.65 points for $\pi_{0.5}$, 4.75 points for SmolVLA, and 4.05 points for X-VLA. The strongest gain appears on the Very Hard split of $\pi_{0.5}$, increasing success from 41.0% to 65.0%.

Sample Efficiency on LIBERO. We further evaluate whether VLA-Corrector can compensate for limited task-specific data. Table 2 compares a fully fine-tuned $\pi_{0.5}$ model with a few-shot fine-tuned counterpart on LIBERO. Starting from the publicly released LeRobot Cadene et al. (2026) few-shot checkpoint `pi05_libero_base`, VLA-Corrector improves the average success rate from 94.00% to 97.80%, even surpassing the fully fine-tuned baseline (96.95%).

These results suggest that few-shot fine-tuning can already learn much of the normal task trajectories, but may lack coverage of drifted states and their recovery behaviors. Instead of improving recovery by exposing the backbone to more rare failure cases during training, VLA-Corrector makes recovery easier at inference time by interrupting error accumulation early and guiding corrective replanning, thereby alleviating the dependence on additional post-training data for robust recovery.

Data efficiency of corrector training. We further examine how much demonstration data is needed to train the external corrector. Since VLA-Corrector only learns local latent-dynamics consistency rather than a full policy or high-capacity world model, it should not require exhaustive task coverage. We therefore first hold out 20% of the original MetaWorld demonstration set for validation, then subsample the remaining training split with ratios $r \in 0.2, 0.4, 0.6, 0.8, 1.0$ and evaluate the resulting correctors under the same $\pi_{0.5}$ backbone. Here, $r = 1.0$ denotes using the full training split after validation hold-out, rather than using all original demonstrations for training.

Table 3 shows that the corrector benefits from more demonstrations but exhibits diminishing returns. Performance already exceeds the baseline with a moderate amount of data and saturates around $r = 0.6$–0.8. This suggests that our lightweight latent-dynamics corrector only needs to capture a local on-track consistency signal, rather than a complete dynamics model, which makes it relatively data-efficient to train.

Table 4 Full performance–efficiency trade-off across VLA backbones and action horizons. Success rate, average policy calls per episode, and relative success-per-call efficiency gains are reported. VLA-Corrector consistently improves task success while maintaining or reducing policy calls in most settings.

Model	Horizon	Baseline		Ours (LVM+OGG)		Improvements		
		Succ (%)↑	Calls ↓	Succ (%)↑	Calls ↓	ΔSucc (%)↑	ΔCalls ↓	Eff. Gain (%)↑
$\pi_{0.5}$	10	64.50	20.41	**72.40**	**17.64**	↑ 7.90	↓ 2.77	**+29.9%**
	20	61.40	10.77	**69.10**	**9.95**	↑ 7.70	↓ 0.82	**+21.8%**
	30	56.10	7.97	**63.50**	**7.92**	↑ 7.40	↓ 0.05	**+13.9%**
	40	54.45	5.96	**60.80**	6.04	↑ 6.35	↑ 0.08	**+10.2%**
	50	48.72	5.15	**58.70**	**4.98**	↑ 9.98	↓ 0.17	**+24.6%**
SmolVLA	10	61.90	19.27	**73.00**	**15.64**	↑ 11.10	↓ 3.63	**+45.3%**
	20	58.90	11.32	**68.90**	**9.50**	↑ 10.00	↓ 1.82	**+39.4%**
	30	58.10	7.90	**65.20**	**7.60**	↑ 7.10	↓ 0.30	**+16.6%**
	40	56.80	5.64	**67.20**	**5.13**	↑ 10.40	↓ 0.51	**+30.0%**
	50	54.20	4.86	**62.90**	**4.68**	↑ 8.70	↓ 0.18	**+20.6%**
X-VLA	4	68.50	46.58	**72.00**	**35.20**	↑ 3.50	↓ 11.38	**+39.1%**
	8	67.80	23.18	**72.10**	**19.48**	↑ 4.30	↓ 3.70	**+26.5%**
	16	55.50	13.71	**60.90**	13.92	↑ 5.40	↑ 0.21	**+8.1%**
	24	52.90	10.13	**59.20**	**9.92**	↑ 6.30	↓ 0.21	**+14.3%**
	32	44.00	8.61	**54.40**	**8.34**	↑ 10.40	↓ 0.27	**+27.6%**

ΔSucc: absolute success-rate improvement. **ΔCalls**: absolute change in policy calls per episode. **Eff. Gain**: relative improvement in success-per-call over the baseline.

Performance–Efficiency Trade-off. VLA-Corrector improves success-per-call efficiency across action horizons rather than simply querying the policy more often. As shown in Figure 4, the largest success-per-call gains reach 29.9% for $\pi_{0.5}$, 45.3% for SmolVLA, and 39.1% for X-VLA. For example, on SmolVLA with horizon 10, success improves from 61.90% to 73.00% while policy calls decrease from 19.27 to 15.64. This suggests that VLA-Corrector makes each policy query more useful by interrupting stale chunks and guiding recovery.

The full horizon sweep shows that VLA-Corrector does not obtain higher success by simply increasing the number of policy queries. Across $\pi_{0.5}$, SmolVLA, and X-VLA, it consistently improves success-per-call efficiency, with especially strong gains at long horizons where stale actions have more time to accumulate errors. This supports the central adaptive-horizon hypothesis: long chunks remain useful during stable execution, but they should be interrupted once the latent visual dynamics indicate drift.

4.2 Mechanism Analysis

In this section, we will analyze the mechanisms behind VLA-Corrector from three perspectives: **1) LVM detection**, whether LVM provides a useful and well-timed drift signal; and **2) OGG correction**, whether OGG improves recovery after an interrupt event; **3) controlled recovery**, whether the full framework improves recovery in a controlled failure case.

LVM Detection. LVM provides an informative drift signal. As shown in Figure 5, successful episodes concentrate at low E_t, while failed episodes show a heavier high-score tail and trigger more interrupt events. This indicates that E_t captures failure-prone visual dynamics mismatch.

LVM also intervenes at meaningful moments. We manually divide MetaWorld trajectories into *critical* phases, such as precise grasping and alignment, and *non-critical* phases, such as tolerant transport after a stable grasp. Figure 6 shows that 83.7% of truncations occur in critical phases. This result verifies the core adaptive-horizon intuition of VLA-Corrector: it does not simply shorten the horizon everywhere, but instead preserves long-horizon efficiency in tolerant phases and restores short-horizon precision when the task enters error-sensitive phases.

OGG Correction. We isolate OGG by comparing standard re-inference and OGG-guided re-inference

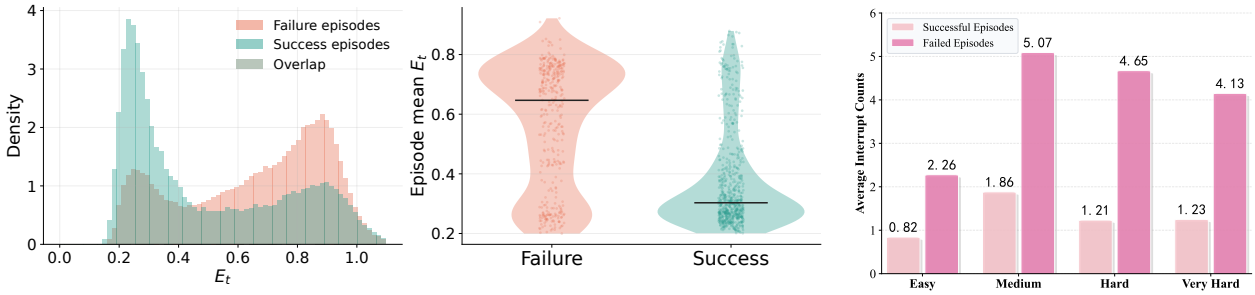
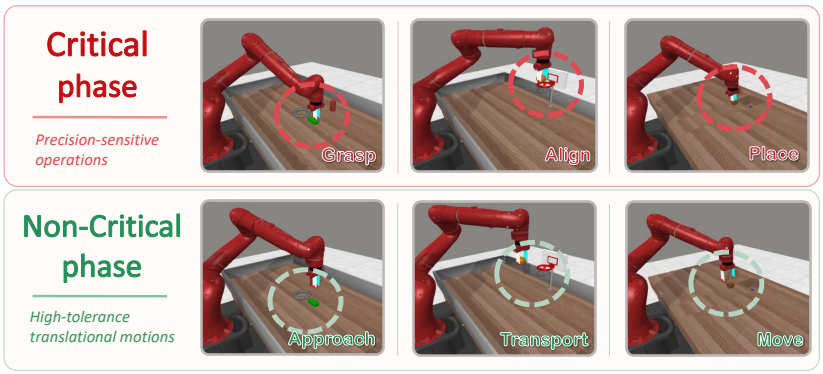


Figure 5 LVM detection analysis. Left: distribution of the inconsistency score E_t, where successful episodes concentrate at low values and failed episodes show a heavier high-score tail. **Right**: interrupt frequency, where failed episodes trigger more interrupt events than successful ones.



Truncation Distribution by Task Phase

Phase	Rate
Critical	**83.7%**
Non-critical	16.3%

5.1× more truncations occur in critical phases than in non-critical phases.

Figure 6 Task-phase analysis of LVM-triggered truncation. We manually divide MetaWorld trajectories into *critical* and *non-critical* phases. **Left**: representative trajectories show that truncation is triggered much more frequently during critical phases. **Right**: 83.7% of truncations occur in critical phases, while only 16.3% occur in non-critical phases.

after the same interrupt-triggered truncation. Recovery is counted when $E_t < T_{\text{off}}$ within the next 10 steps. As shown in Fig. 7, OGG improves recovery across all difficulty levels, with an average gain of 0.23. This indicates that, after truncation stops stale actions, OGG-guided re-inference further improves the quality of the recovery replan compared to standard re-inference.

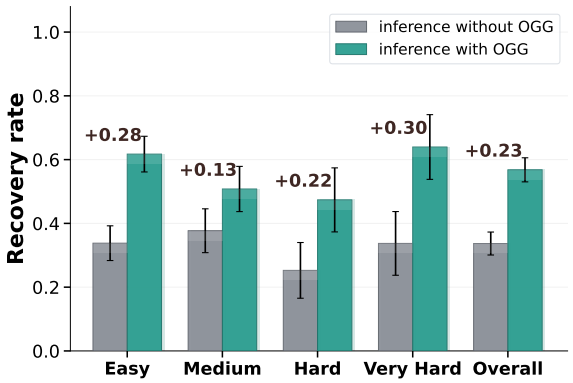


Figure 7 Post-interrupt recovery. OGG-guided inference consistently outperforms standard inference.

Controlled Recovery Case. Figure 8 shows a representative LIBERO episode where both methods start from the same initial state and encounter the same grasping error when approaching the cup handle. In the top row, the baseline rollout is only monitored by LVM for error alignment without truncating the remaining actions; after the deviation is detected, it continues executing the original action chunk, leading to an unstable grasp, cup drop, and task failure. In the bottom row, VLA-Corrector detects the same deviation but actively truncates the stale actions and triggers corrective replanning, allowing the robot to recover a stable grasp and place the cup at the target location. This controlled comparison highlights the overall effectiveness of VLA-Corrector under the same initial condition and the same detected execution error.

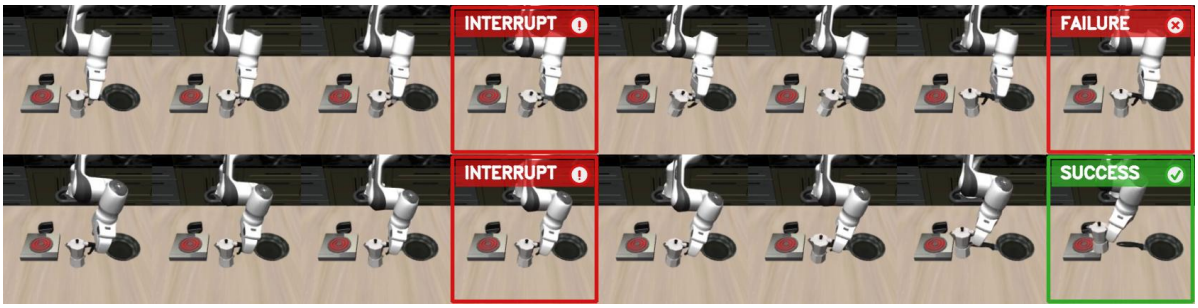


Figure 8 Controlled recovery case. Given the same initial state and detected grasping error, the monitored baseline continues the original chunk and fails (**top**), while VLA-Corrector truncates stale actions, replans with OGG, and completes the task (**bottom**).

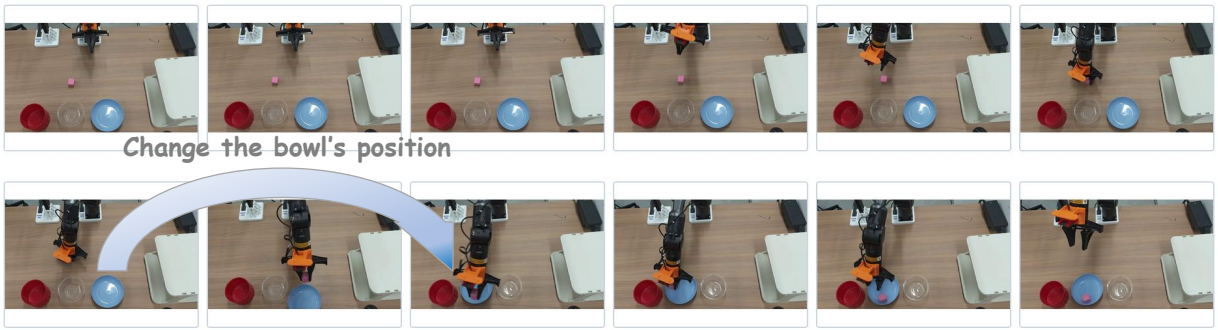


Figure 9 Real-world disturbance recovery demo. A human shifts the blue bowl during execution, requiring the robot to recover from an outdated action chunk.

4.3 Real-World Evaluation

Setup. We evaluate VLA-Corrector on an AgileX PiPER 6-DoF arm using $\pi_{0.5}$ as the backbone. The benchmark includes three task groups, each with three tasks and 20 trials: **1) Pick-and-place** tests standard manipulation, **2) Alignment** tests precision-sensitive placement or insertion, and **3) Disturbance recovery** tests recovery when objects or targets are manually shifted during execution. Figure 9 shows a representative disturbance recovery demo, where the target bowl is shifted during execution. Full task definitions, protocols, failure analysis, and additional demos are provided in Appendix E.

Table 5 Real-world evaluation on AgileX PiPER. Success rate (%, ↑) over three task groups. Each group contains three tasks with 20 trials per task. We report 95% binomial confidence intervals.

Method	Pick-place↑	Alignment↑	Disturbance↑	Avg.↑
$\pi_{0.5}$ Baseline	70.0 ± 11.6	56.7 ± 12.5	40.0 ± 12.4	55.6 ± 7.3
+ VLA-Corrector	**78.3 ± 10.4** ↑ 8.3	**73.3 ± 11.2** ↑ 16.6	**68.3 ± 11.8** ↑ 28.3	**73.3 ± 6.5** ↑ 17.7

Results. Table 5 shows that VLA-Corrector improves all three task groups, increasing average success from 55.6% to 73.3%. The gain is modest on pick-and-place (+8.3), larger on alignment (+16.6), and largest on disturbance recovery (+28.3). This trend matches the intended role of VLA-Corrector: it preserves standard execution performance, improves precision-sensitive manipulation, and is most beneficial when online disturbances make the remaining action chunk outdated.

4.4 Ablation Studies

We ablate VLA-Corrector on $\pi_{0.5}$ in MetaWorld, focusing on **(1) components**: necessity of truncation and OGG; **(2) detector design**: decoupling monitoring from backbone fine-tuning; and **(3) sensitivity**, with detailed results in Appendix D.1.

Table 6 Component ablation on MetaWorld. Success rate (%, ↑). Green arrows indicate average improvement over the baseline.

Variant	Easy↑	Medium↑	Hard↑	Very Hard↑	Avg.↑
Baseline (Open-loop)	70.5	45.0	38.3	41.0	48.70
+ Truncation Only	81.8	53.6	**50.0**	56.0	60.35 ↑ 11.65
+ Truncation + OGG	**83.2**	**61.7**	47.5	**65.0**	**64.35** ↑ 15.65

Table 7 Coupled vs. decoupled detection on MetaWorld. Success rate (%, ↑). Both variants use OGG; only the detector design differs.

Detector	Easy	Medium	Hard	Very Hard	Avg.
Internal Head + OGG	65.2	46.0	36.5	50.5	49.55
Decoupled LVM + OGG	**83.2** ↑ 18.0	**61.7** ↑ 15.7	**47.5** ↑ 11.0	**65.0** ↑ 14.5	**64.35** ↑ 14.80

Component effects. Table 6 shows that truncation alone raises average success from 48.70% to 60.35%, confirming the importance of stopping stale actions. Adding OGG further improves the average to 64.35%, showing that post-truncation recovery also benefits from guided re-inference.

Decoupled vs. coupled detection. We next evaluate whether the drift detector should be decoupled from the VLA backbone. For the coupled variant, we fine-tune $\pi_{0.5}$ with an additional auxiliary head that predicts the same short-horizon latent residual as LVM. The head uses the final-layer hidden state of the last token together with the executed action. Both variants use OGG after detected interruptions; only the detector design differs.

Table 7 shows that the decoupled LVM achieves a higher average success rate than the coupled internal detector under the same OGG setting, improving from 49.55% to 64.35%. A possible reason is that the internal auxiliary objective updates backbone representations that are also used for VLM-to-action planning, which may negatively affect the original action-generation behavior. In contrast, the external LVM learns the monitoring signal on frozen VLA features, avoiding direct modification of the policy representation.

We further analyze the sensitivity of VLA-Corrector to OGG guidance strength and LVM capacity. The results show that moderate guidance strength and a 40M LVM are sufficient: overly strong guidance can degrade harder tasks, while increasing the monitor size beyond 40M brings little additional gain. Detailed information is provided in Appendix D.

5 Conclusion

This paper studies the open-loop blind spot in action-chunked VLA policies, where fixed horizons improve policy-call efficiency but allow stale actions to accumulate errors. VLA-Corrector addresses this issue with a lightweight detect-and-correct layer that monitors latent visual dynamics, truncates stale actions when drift persists, and guides the next inference toward recovery. Our results show that small inference-time modules can provide targeted robustness gains without retraining the VLA backbone. Rather than replacing action chunking, VLA-Corrector makes it adaptive: long-horizon execution is preserved when reliable, while corrective replanning is invoked when the current chunk should no longer be trusted.

References

Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, and 1 others. 2025. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, and 1 others. 2024. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*.

Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, and 1 others. 2026. Lerobot: An open-source library for end-to-end robot learning. *arXiv preprint arXiv:2602.22818*.

Jun Cen, Siteng Huang, Yuqian Yuan, Kehan Li, Hangjie Yuan, Chaohui Yu, Yuming Jiang, Jiayan Guo, Xin Li, Hao Luo, and 1 others. 2025. Rynnvla-002: A unified vision-language-action and world model. *arXiv preprint arXiv:2511.17502*.

Hao Chen, Jiaming Liu, Chenyang Gu, Zhuoyang Liu, Renrui Zhang, Xiaoqi Li, Xiao He, Yandong Guo, Chi-Wing Fu, Shanghang Zhang, and 1 others. 2025. Fast-in-slow: A dual-system foundation model unifying fast manipulation within slow reasoning. *arXiv preprint arXiv:2506.01953*.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2025. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704.

Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, and 1 others. 2025. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. *arXiv preprint arXiv:2505.23705*.

Yichao Fu, Xuewei Wang, Yuandong Tian, and Jiawei Zhao. 2025. Deep think with confidence. *arXiv preprint arXiv:2508.15260*.

George Jiayuan Gao, Tianyu Li, and Nadia Figueroa. 2025. Out-of-distribution recovery with object-centric keypoint inverse policy for visuomotor imitation learning. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9816–9828. IEEE.

Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, and 1 others. 2024. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*.

Ruiyu Gou. 2024. *Learning temporal action chunking for motor control*. Ph.D. thesis, University of British Columbia.

Zhi Hou, Tianyi Zhang, Yuwen Xiong, Haonan Duan, Hengjun Pu, Ronglei Tong, Chengyang Zhao, Xizhou Zhu, Yu Qiao, Jifeng Dai, and 1 others. 2025. Dita: Scaling diffusion transformer for generalist vision-language-action policy. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7686–7697.

Zheyuan Hu, Robyn Wu, Naveen Enock, Jasmine Li, Riya Kadakia, Zackory Erickson, and Aviral Kumar. 2025. Rac: Robot learning for long-horizon tasks by scaling recovery and correction. *arXiv preprint arXiv:2509.07953*.

Nils Ingelhag, Jesper Munkeby, Michael C Welle, Marco Moletta, and Danica Kragic. 2025. Real-time operator takeover for visuomotor diffusion policy training. *arXiv preprint arXiv:2502.02308*.

Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, and 1 others. 2025. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*.

Arnav Kumar Jain, Vibhakar Mohta, Subin Kim, Atiksh Bhardwaj, Juntao Ren, Yunhai Feng, Sanjiban Choudhury, and Gokul Swamy. 2025. A smooth sea never made a skilled sailor: Robust imitation via learning to search. *arXiv preprint arXiv:2506.05294*.

Sunshine Jiang, Xiaolin Fang, Nicholas Roy, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Siddharth Ancha. 2025a. Streaming flow policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories. *arXiv preprint arXiv:2505.21851*.

Yuming Jiang, Siteng Huang, Shengke Xue, Yaxi Zhao, Jun Cen, Sicong Leng, Kehan Li, Jiayan Guo, Kexiang Wang, Mingxiu Chen, and 1 others. 2025b. Rynnvla-001: Using human demonstrations to improve robot manipulation. *arXiv preprint arXiv:2509.15212*.

Dong Jing, Gang Wang, Jiaqi Liu, Weiliang Tang, Zelong Sun, Yunchao Yao, Zhenyu Wei, Yunhui Liu, Zhiwu Lu, and Mingyu Ding. 2025. Mixture of horizons in action chunking. *arXiv preprint arXiv:2511.19433*.

Sarosh Khan and Ellie Tanimura. 2025. Test-time stochasticity estimation for adaptive action chunk selection.

Moo Jin Kim, Chelsea Finn, and Percy Liang. 2025. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, and 1 others. 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*.

Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang Nie. 2025. Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification. *arXiv preprint arXiv:2508.21046*.

Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, and 1 others. 2023. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*.

Yuanchang Liang, Xiaobo Wang, Kai Wang, Shuo Wang, Xiaojiang Peng, Haoyu Chen, David Kim Huat Chua, and Prahlad Vadakkepat. 2026. Adaptive action chunking at inference-time for vision-language-action models. *arXiv preprint arXiv:2604.04161*.

Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. 2022. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791.

Isabella Liu, An-Chieh Cheng, Rui Yan, Geng Chen, Ri-Zhao Qiu, Xueyan Zou, Sha Yi, Hongxu Yin, Xiaolong Wang, and Sifei Liu. 2026. Long-horizon manipulation via trace-conditioned vla planning. *arXiv preprint arXiv:2604.21924*.

Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. 2024. Bidirectional decoding: Improving action chunking via guided test-time sampling. *arXiv preprint arXiv:2408.17355*.

Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. 2024. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2405.14093*.

Cyrus Neary, Omar G Younis, Artur Kuramshin, Ozgur Aslan, and Glen Berseth. 2025. Improving pre-trained vision-language-action policies with model-based search. *arXiv preprint arXiv:2508.12211*.

Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, and 1 others. 2024. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE.

Minho Park, Kinam Kim, Junha Hyung, Hyojin Jang, Hoiyeong Jin, Jooyeol Yun, Hojoon Lee, and Jaegul Choo. 2025. Acg: Action coherence guidance for flow-based vla models. *arXiv preprint arXiv:2510.22201*.

Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. 2025. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*.

Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. 2025. Leave no observation behind: Real-time correction for vla action chunks. *arXiv preprint arXiv:2509.23224*.

Archit Sharma, Ahmed M Ahmed, Rehaan Ahmad, and Chelsea Finn. 2023. Self-improving robots: End-to-end autonomous visuomotor reinforcement learning. *arXiv preprint arXiv:2303.01488*.

Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, and 1 others. 2025. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*.

Junhyuk So, Chiwoong Lee, Shinyoung Lee, Jungseul Ok, and Eunhyeok Park. 2025. Improving generative behavior cloning via self-guidance and adaptive chunking. *arXiv preprint arXiv:2510.12392*.

Haoming Song, Delin Qu, Yuanqi Yao, Qizhi Chen, Qi Lv, Yiwen Tang, Modi Shi, Guanghui Ren, Maoqing Yao, Bin Zhao, and 1 others. 2025a. Hume: Introducing system-2 thinking in visual-language-action model. *arXiv preprint arXiv:2505.21432*.

Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao, Wei Zhao, Zhide Zhong, Zongyuan Ge, Jun Ma, and Haoang Li. 2025b. Accelerating vision-language-action model integrated with action chunking via parallel decoding. *arXiv preprint arXiv:2503.02310*.

Zhanyi Sun and Shuran Song. 2025. Latent policy barrier: Learning robust visuomotor policies by staying in-distribution. *arXiv preprint arXiv:2508.05941*.

Libo Wang. 2025. Sigma: The key for vision-language-action models toward telepathic alignment. *arXiv preprint arXiv:2512.00783*.

Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, and 1 others. 2026a. Vla-adapter: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI conference on artificial intelligence*, volume 40, pages 18638–18646.

Zhendong Wang, Zhaoshuo Li, Ajay Mandlekar, Zhenjia Xu, Jiaojiao Fan, Yashraj Narang, Linxi Fan, Yuke Zhu, Yogesh Balaji, Mingyuan Zhou, and 1 others. 2024. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. *arXiv preprint arXiv:2410.21257*.

Zihua Wang, Zhitao Lin, Ruibo Li, Yu Zhang, Xu Yang, Siya Mi, and Xiu-Shen Wei. 2026b. Open-loop planning, closed-loop verification: Speculative verification for vla. *arXiv preprint arXiv:2604.02965*.

Junjie Wen, Yichen Zhu, Minjie Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Xiaoyu Liu, Chaomin Shen, Yaxin Peng, and Feifei Feng. 2025. Diffusionvla: Scaling robot foundation models via unified diffusion and autoregression. In *Forty-second International Conference on Machine Learning*.

Pengyuan Wu, Pingrui Zhang, Zhigang Wang, Dong Wang, Bin Zhao, and Xuelong Li. 2026a. Closed-loop action chunks with dynamic corrections for training-free diffusion policy. *arXiv preprint arXiv:2603.01953*.

Zhichao Wu, Junyin Ye, Zhilong Zhang, Yihao Sun, Haoxin Lin, Jiaheng Luo, Haoxiang Ren, Lei Yuan, and Yang Yu. 2026b. Speedup patch: Learning a plug-and-play policy to accelerate embodied manipulation. *arXiv preprint arXiv:2603.20658*.

Changhua Xu, Jie Lu, Junyu Xuan, and En Yu. 2026. Vgas: Value-guided action-chunk selection for few-shot vision-language-action adaptation. *arXiv preprint arXiv:2602.07399*.

Shaoqing Xu, Fang Li, Zhixiang Duan, Yifan Yang, Tianshi Xie, and Zhi-Xin Yang. Vla-in-the-loop: Online policy correction with world models for robust robotic grasping.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. 2020. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on robot learning*, pages 1094–1100. PMLR.

Thomas TCK Zhang, Daniel Pfrommer, Chaoyi Pan, Nikolai Matni, and Max Simchowitz. Action chunking and data augmentation yield exponential improvements in behavior cloning for continuous spaces. In *The Fourteenth International Conference on Learning Representations*.

Chen Zhao, Zhuoran Wang, Haoyang Li, Shifeng Bao, Guanlin Li, Youhe Feng, Yang Li, Jie Tang, and Jing Zhang. 2026. Action draft and verify: A self-verifying framework for vision-language-action model. *arXiv preprint arXiv:2603.18091*.

Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, and 1 others. 2025. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713.

Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023a. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023b. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*.

Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, and 1 others. 2025. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*.

Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, and 1 others. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR.

Appendix

A Related Work

A.1 Generative VLA Models

Embodied intelligence is shifting from task-specific policies toward unified Vision-Language-Action (VLA) foundation models. Representative works have shown that combining large-scale vision-language pretraining with robot control can substantially improve cross-task generalization, semantic understanding, and cross-platform transfer O'Neill et al. (2024); Black et al. (2024); Zitkovich et al. (2023); Kim et al. (2024); Ghosh et al. (2024); Li et al. (2023); Bjorck et al. (2025); Pertsch et al. (2025); Song et al. (2025b). On the action modeling side, modern VLA and general robot policies often adopt generative frameworks to model high-dimensional, continuous, and multi-modal action distributions: diffusion models generate actions through iterative denoising Chi et al. (2025); Wen et al. (2025); Black et al. (2024); Chen et al. (2025); Li et al. (2025); Zhao et al. (2025); Hou et al. (2025); Kim et al. (2025); Song et al. (2025a), while flow matching directly learns velocity fields along continuous trajectories. These approaches typically offer stronger action expressivity, but they also incur higher policy-call costWang et al. (2024). Consequently, reducing control latency without sacrificing generation quality has become a central challenge in deploying VLA models.

A.2 Action Chunk and Horizon Trade-off

To alleviate the tension between high-frequency control and expensive policy inference, **action chunk** has been widely adopted in continuous control. The core idea is to predict multiple future actions in a single inference call and then execute them sequentially with a downstream controller Chi et al. (2025); Zhao et al. (2023b); Black et al. (2024); Shukor et al. (2025); Bjorck et al. (2025); Wang (2025); Driess et al. (2025). This design effectively amortizes policy-call frequency and improves temporal smoothness, but it also reduces the use of fresh observations within a fixed action horizon. A longer horizon further lowers inference frequency and maintains action consistency, but weakens responsiveness to environmental changes; a shorter horizon more closely resembles strict closed-loop control and often provides better local recovery, yet requires much more frequent policy calls Jing et al. (2025); Zhang et al.; Liu et al. (2024); Khan and Tanimura (2025); So et al. (2025); Fu et al. (2025); Zhao et al. (2026); Liu et al. (2026). Most existing approaches therefore seek a compromise under a static horizon, whereas our work focuses on an event-triggered dynamic horizon: the system retains the efficiency of long chunks during stable phases and truncates them on demand to recover closed-loop responsiveness during local failures.

A.3 Failure Recovery in Visuomotor Policies

Long-horizon visuomotor control commonly suffers from compounding errors and covariate shift: once execution drifts away from the training distribution, the policy often cannot recover on its own. Existing approaches can be broadly divided into two categories. One line of work expands the training distribution through additional interaction, human intervention, or recovery data collection, and then updates the policy accordingly Sharma et al. (2023); Ingelhag et al. (2025); Hu et al. (2025); Neary et al. (2025). Another line explicitly introduces recovery policies, search mechanisms, or distributional constraints to pull the system back to a safe region once it departs from the expert manifold Gao et al. (2025); Sun and Song (2025); Jain et al. (2025); Liang et al. (2026); Gou (2024); Wu et al. (2026b); Xu et al.. These works collectively show that explicit recovery mechanisms can substantially improve the robustness of visuomotor policies; however, they typically rely on additional recovery data, separately trained recovery modules, or joint optimization tailored to a specific policy backbone.

B Method Details

B.1 Details of Event-Triggered Truncation

This section provides the full event-triggered truncation rule used by LVM. Given the online inconsistency score E_t, the monitor maintains a sliding window of recent scores,

$$\mathbf{E}_W = \{E_{t-w+1}, \ldots, E_t\}.$$

Let

$$M_e = \text{median}(\mathbf{E}_W), \qquad \text{MAD} = \text{median}(|E_i - M_e|), \quad E_i \in \mathbf{E}_W.$$

We use the median absolute deviation because it is less sensitive to transient score spikes than the mean and variance. Based on this robust statistic, we define asymmetric on/off thresholds,

$$T_{\text{on}} = M_e + \lambda_{\text{on}}\text{MAD}, \qquad T_{\text{off}} = M_e + \lambda_{\text{off}}\text{MAD}, \qquad \lambda_{\text{on}} > \lambda_{\text{off}}.$$

The higher threshold T_{on} is used to confirm abnormal deviation, while the lower threshold T_{off} provides hysteresis and prevents rapid oscillation between normal and abnormal states.

To avoid triggering on isolated spikes, we maintain a persistence counter:

$$c_t = \begin{cases} c_{t-1} + 1, & \text{if } E_t > T_{\text{on}}, \\ 0, & \text{if } E_t < T_{\text{off}}, \\ c_{t-1}, & \text{otherwise.} \end{cases} \tag{12}$$

An interrupt event is triggered when

$$c_t \geq p, \tag{13}$$

where p is the patience parameter. After an interrupt event, the remaining actions in the current execution queue are discarded, the counter is reset, and the next policy query is performed under corrective mode. If h actions have already been executed from the current queue, the realized action horizon is

$$H_{\text{adaptive}} = h < H.$$

In this way, the system preserves long-horizon execution during stable phases, while shortening the horizon when persistent visual drift indicates that the current action chunk is no longer reliable.

B.2 LVM and OGG Runtime Parameters

LVM monitoring parameters. The Latent-space Vision Monitor uses a robust online thresholding state machine. We maintain a sliding window of recent inconsistency scores with window size 15. The dynamic thresholds are computed using $\lambda_{\text{on}} = 3.0$ and $\lambda_{\text{off}} = 2.0$. An interrupt event is triggered only when the inconsistency score exceeds the activation threshold for $p = 5$ consecutive steps. Similarly, the monitor requires 5 consecutive safe steps for reset. To avoid repeated triggers immediately after intervention, we use an interrupt cooldown of 10 steps.

OGG parameters. Online Gradient Guidance is invoked only for the single policy query immediately following an interrupt-triggered truncation. When an interrupt event occurs, the remaining action queue is cleared, the next replan is marked as a guidance replan, and OGG is applied during that next action-chunk generation. Subsequent policy calls return to standard inference unless another interrupt event is detected. The guidance strength is controlled by η. In our experiments, we use $\eta = 1$ as the default setting for $\pi_{0.5}$, SmolVLA, and X-VLA. The sensitivity of performance to η is analyzed in Table 8.

C Training and Implementation Details

C.1 Benchmarks and Evaluation Protocol

We evaluate VLA-Corrector on two simulation benchmarks: MetaWorld Yu et al. (2020) and LIBERO Liu et al. (2023). MetaWorld tests contact-rich manipulation robustness across difficulty splits, while LIBERO

evaluates language-conditioned long-horizon task execution. We use $\pi_{0.5}$ Intelligence et al. (2025) as the main backbone, and further test SmolVLA Shukor et al. (2025) and X-VLA Zheng et al. (2025) for cross-architecture evaluation. We report task success rate as the primary metric, together with policy calls, success-per-call efficiency, and post-interrupt recovery rate when applicable.

For MetaWorld, tasks are grouped into four difficulty splits: *Easy*, *Medium*, *Hard*, and *Very Hard*. All online evaluations are conducted task by task within each split, and we report the average success rate over tasks in the corresponding split. Unless otherwise specified, each task is evaluated with 20 episodes. For the corrector data-scaling study, we use a fixed episode-level split before subsampling the training data: 10% of episodes are held out for validation, 10% are held out for testing, and the remaining episodes form the training pool. We then subsample this training pool according to the training ratio r. In the multi-seed data-scaling analysis, we use split seeds $\{42, 123, 999\}$ and require at least 20 episodes in the train, validation, and test partitions. All simulation experiments are conducted on 8 NVIDIA A100-SXM4-40GB GPUs.

C.2 External Corrector Architecture

The external corrector M_ϕ is implemented as a residual MLP for short-horizon latent dynamics prediction. We first map the executed action through a linear embedding layer, concatenate the resulting action feature with the current visual latent Z_t^{real}, and feed the combined representation into a stack of residual MLP blocks. The network predicts the short-horizon latent residual rather than the absolute future latent state. In all experiments, we use a four-layer hidden configuration with width $[2048, 2048, 2048, 2048]$. The exact number of parameters varies slightly across benchmarks and VLA backbones because both the action dimensionality and the visual latent dimensionality differ across settings. Overall, the correctors used in our experiments contain approximately 38–42M parameters, and we refer to this external module as a lightweight ∼40M MLP corrector.

C.3 Corrector Training

The corrector is trained after the VLA backbone has been fine-tuned on the benchmark training set. We freeze the VLA backbone and use its visual encoder to extract latent representations from demonstration trajectories. The corrector is then trained on these frozen latents to predict the short-horizon visual latent residual induced by the executed action. We use AdamW as the optimizer with learning rate 3×10^{-4} and weight decay 10^{-4}. The learning rate is scheduled by cosine annealing with $\eta_{\min} = 0.01 \times \text{lr}$. We train for 30 epochs with early stopping patience 5. For the ratio-sweep training runs, we use batch size 128 and evaluation batch size 256. For the deployed h1-k10 correctors used in evaluation, we use batch size 512, train for 30 epochs, and optimize a cosine-based training loss. The number of optimization steps depends on the number of training samples under each data ratio:

$$\text{steps per epoch} = \left\lceil \frac{N_{\text{train}}}{B} \right\rceil, \qquad \text{total steps} = 30 \times \text{steps per epoch},$$

where N_{train} is the number of training samples and B is the batch size.

C.4 Compute Resources

All simulation experiments are conducted on NVIDIA A100-SXM4-40GB GPUs. The main training and evaluation jobs are run on a server with 8 such GPUs. Corrector training is lightweight compared with VLA backbone fine-tuning because it only optimizes the external MLP on frozen VLA features. During online evaluation, LVM monitoring adds only a forward pass through the external corrector, while OGG introduces additional gradient computation only for the single recovery query following an interrupt event.

D Additional Experimental Results

D.1 Ablation Sensitivity

We provide additional ablations on OGG guidance strength, and LVM capacity. All experiments are conducted with $\pi_{0.5}$ on MetaWorld.

Table 8 Ablation on OGG guidance strength η. Success rate (%, ↑) on MetaWorld. The highlighted row is the default setting.

Guidance Strength	Easy↑	Medium↑	Hard↑	Very Hard↑	Avg.↑
$\eta = 0.1$	82.7	56.8	45.2	**66.0**	62.68
$\eta = 1$ **(default)**	**83.2** ↑ 0.5	**61.7** ↑ 4.9	**47.5** ↑ 2.3	65.0 ↓ 1.0	**64.35** ↑ 1.67
$\eta = 10$	82.5	56.8	38.3	65.0	60.65 ↓ 3.70
$\eta = 100$	80.9	55.0	36.7	63.0	58.90 ↓ 5.45

Table 9 Ablation on LVM capacity. Success rate (%, ↑) on MetaWorld. The highlighted row is the default setting.

Monitor Capacity	Easy↑	Medium↑	Hard↑	Very Hard↑	Avg.↑
LVM-10M	78.5	52.4	39.8	55.6	56.58
LVM-40M (default)	**83.2** ↑ 4.7	61.7 ↑ 9.3	47.5 ↑ 7.7	**65.0** ↑ 9.4	**64.35** ↑ 7.77
LVM-160M	82.8 ↓ 0.4	**62.1** ↑ 0.4	**48.0** ↑ 0.5	64.2 ↓ 0.8	64.28 ↓ 0.07

Table 10 Cross-domain corrector generalization on MetaWorld. Success rate (%, ↑) of the same $\pi_{0.5}$ MetaWorld baseline equipped with correctors trained from different demonstration domains. Green arrows indicate absolute improvement over the baseline.

Method	Easy↑	Medium↑	Hard↑	Very Hard↑	Avg.↑
$\pi_{0.5}$ Baseline	70.5	45.0	38.3	41.0	48.7
+ LIBERO-trained Corrector	70.9 ↑ 0.4	50.5 ↑ 5.5	40.7 ↑ 2.4	45.0 ↑ 4.0	51.8 ↑ 3.1
+ MetaWorld-trained Corrector	**74.1** ↑ 3.6	**53.8** ↑ 8.8	**52.9** ↑ 14.6	**54.0** ↑ 13.0	**58.7** ↑ 10.0

Tables 8 and 9 show that VLA-Corrector does not require overly strong guidance or excessively large monitors. $\eta = 1$ achieves the best average success, while larger guidance weakens performance on harder tasks. Increasing LVM capacity from 10M to 40M substantially improves success, but 160M provides almost no additional average gain.

D.2 Cross-Domain Corrector Generalization

To examine whether the external corrector learns a transferable latent-dynamics signal rather than only memorizing benchmark-specific trajectories, we evaluate cross-domain corrector transfer on MetaWorld. We use the same $\pi_{0.5}$ MetaWorld baseline policy and compare two VLA-Corrector variants: one using a corrector trained on LIBERO demonstrations, and the other using a corrector trained on MetaWorld demonstrations. Both are evaluated on MetaWorld under the same protocol.

Table 10 shows that the LIBERO-trained corrector still improves the MetaWorld baseline from 48.7% to 51.8%, suggesting that the learned latent-dynamics consistency signal has limited but non-trivial cross-domain transfer. However, the MetaWorld-trained corrector achieves a much larger gain, improving the average success rate to 58.7%. This indicates that while the corrector can capture partially transferable notions of on-track visual dynamics, domain-matched demonstrations remain important for accurate deviation detection and effective corrective guidance.

D.3 Inference-Time Overhead

VLA-Corrector improves policy-call efficiency in the main experiments, but it also introduces extra wall-clock computation when OGG is activated because OGG performs online gradient computation during recovery inference. We therefore report detailed inference-time statistics in MetaWorld under the default evaluation setting of each backbone. The "w/o OGG" timing measures the same action-chunk inference pipeline with OGG disabled, while the "w/ OGG" timing includes the additional guidance computation triggered after interrupt events.

Table 11 summarizes the overall timing by backbone. Across all backbones, VLA-Corrector increases wall-clock inference time by 1.62×–1.68× compared with the same pipeline without OGG. This overhead is expected

Table 11 Overall inference-time summary by backbone on MetaWorld. Timing is accumulated over all evaluated episodes and difficulty splits. "w/o OGG" disables guidance while keeping the same action-chunk inference pipeline. The ratio reports the wall-clock overhead of enabling OGG; lower is better (↓).

Backbone	Episodes	Steps / Ep.	Calls / Ep.	w/o OGG s / Ep.↓	w/ OGG s / Ep.↓	Overhead↓
$\pi_{0.5}$	1000	163.77	6.22	2.49	4.03	1.62×
SmolVLA	1000	158.75	5.67	2.13	3.51	1.65×
X-VLA	1000	177.92	10.30	1.54	2.59	1.68×
Overall	3000	166.81	7.39	2.06	3.38	**1.64**×

Table 12 Average per-step inference-time overhead. We report the average inference time per executed environment step. Although OGG-guided recovery queries are slower than standard inference, they are only invoked after interrupt events, so the amortized per-step overhead remains small.

Backbone	w/o OGG ms / Step↓	w/ OGG ms / Step↓	Extra ms / Step↓	Relative Overhead↓
$\pi_{0.5}$	15.23	24.62	+9.39	1.62×
SmolVLA	13.44	22.14	+8.70	1.65×
X-VLA	8.66	14.55	+5.89	1.68×
Overall	12.32	20.25	**+7.93**	**1.64**×

Table 13 Average inference time per MetaWorld task. Each task is evaluated with 20 episodes. The table reports the average total inference time per task and the average number of OGG recovery events per task.

Backbone	Tasks	Episodes / Task	w/o OGG s / Task↓	w/ OGG s / Task↓	OGG Events / Task
$\pi_{0.5}$	50	20	49.87	80.63	71.28
SmolVLA	50	20	42.66	70.29	62.24
X-VLA	50	20	30.81	51.76	119.76

because OGG is gradient-based. Importantly, the extra computation is event-triggered: it is only applied to recovery queries after an interrupt event, rather than to every policy call.

Table 12 reports the average inference cost per executed environment step. Although OGG requires online gradient computation during recovery queries, its cost is amortized over the full rollout because it is only invoked after detected interrupt events. Averaged over all backbones, enabling OGG increases the per-step inference time from 12.32 ms to 20.25 ms, adding only 7.93 ms per executed step on average.

Table 13 reports the same timing from a per-task perspective. Each MetaWorld task is evaluated with 20 episodes. On average, enabling OGG adds 30.76 seconds per task for $\pi_{0.5}$, 27.63 seconds for SmolVLA, and 20.95 seconds for X-VLA. This cost corresponds to 71.28, 62.24, and 119.76 OGG recovery events per task, respectively.

Discussion. These results show that OGG is the main source of additional wall-clock inference time. A standard action-chunk inference takes 278.01 ms on average, while an OGG-guided recovery query takes 588.52 ms, about 2.12× slower. However, this overhead should be interpreted together with the event-triggered nature of the method. VLA-Corrector does not replace all policy calls with gradient-guided inference; it uses standard chunked execution during stable phases and only invokes OGG when LVM detects persistent drift. Therefore, the additional computation is paid mainly at recovery moments, where it is exchanged for higher success rate and better post-interrupt recovery.

E Real-World Details

E.1 Robot Platform and Task Suite

Robot platform. All real-world experiments are conducted on an **AgileX PiPER** 6-DoF robotic arm. We use $\pi_{0.5}$ as the action-chunked VLA baseline and evaluate VLA-Corrector on top of the same fine-tuned backbone.

Table 14 Real-world task suite on AgileX PiPER. The benchmark contains three task groups and nine tasks in total. The disturbance group reuses manipulation skills from the first two groups but introduces online pose changes during precision-sensitive phases.

Group	Task	Description
Pick-and-place	Cube-to-region	Pick up a cube and place it inside a marked target region.
	Cube-to-drawer-top	Pick up a cube and place it on top of a small drawer platform.
	Object-to-container	Pick up a cube and place it into a specific bowl.
Alignment	Corner placement	Place a cube precisely at the upper-left corner of the drawer top.
	Square-hole insertion	Align and insert a small cube into a square hole or slot.
	Edge alignment	Align an object with a narrow marked edge or target strip.
Disturbance recovery	Moving-object grasp	Move the target object when the gripper is about to grasp it.
	Moving-placement target	Move the placement region after the object is grasped.
	Moving-insertion target	Move the square hole or alignment target before insertion.

Both methods share the same RGB camera observation, language instruction, action horizon, and initial task conditions. Each task is evaluated with 20 trials per method. A trial is counted as successful if the robot completes the specified task without human reset after execution starts.

Task design. We design three groups of real-world tasks with increasing difficulty. The first group evaluates standard pick-and-place behavior, where the original action chunk often remains valid. The second group evaluates precise alignment, where small accumulated errors can cause missed placement, collision, or failed insertion. The third group introduces online human disturbances during precision-sensitive phases, directly testing whether VLA-Corrector can recover when the remaining action chunk becomes stale. Table 14 summarizes the task definitions.

E.2 Per-Task Real-World Results

Table 15 reports per-task success rates. The gains are modest on pick-and-place tasks because the baseline can often complete the task when the object and target remain static. The gains become larger on alignment tasks, where small pose errors are less tolerable. The largest improvement appears in disturbance recovery tasks, where the current action chunk often becomes stale after the object or target is manually moved.

Table 15 Per-task real-world success rates. Success rate (%, ↑) over 20 trials per task on the AgileX PiPER 6-DoF arm. Group and overall rows report mean ± standard deviation across tasks. Green arrows indicate absolute improvement over the $\pi_{0.5}$ baseline.

Group	Task	$\pi_{0.5}$ Baseline↑	+ VLA-Corrector↑
Pick-and-place	Cube-to-region	75.0	**85.0** ↑ 10.0
	Cube-to-drawer-top	70.0	**80.0** ↑ 10.0
	Object-to-container	65.0	**70.0** ↑ 5.0
	Group Avg.	**70.0 ± 5.0**	**78.3 ± 7.6** ↑ 8.3
Alignment	Corner placement	60.0	**75.0** ↑ 15.0
	Square-hole insertion	50.0	**70.0** ↑ 20.0
	Edge alignment	60.0	**75.0** ↑ 15.0
	Group Avg.	**56.7 ± 5.8**	**73.3 ± 2.9** ↑ 16.6
Disturbance recovery	Moving-object grasp	45.0	**75.0** ↑ 30.0
	Moving-placement target	40.0	**70.0** ↑ 30.0
	Moving-insertion target	35.0	**60.0** ↑ 25.0
	Group Avg.	**40.0 ± 5.0**	**68.3 ± 7.6** ↑ 28.3
Overall	**Avg. across 9 tasks**	**55.6 ± 13.8**	**73.3 ± 7.1** ↑ 17.7

E.3 Real-World Protocol and Task Details

Evaluation protocol. For each real-world task, we collect 10–30 human teleoperated demonstration trajectories and use them to fine-tune the $\pi_{0.5}$ backbone for that task. The same fine-tuned backbone is then used by both the baseline and VLA-Corrector, so the comparison isolates the effect of the inference-time correction module. For each task and each method, we run 20 evaluation trials. Initial object poses are randomized within a small predefined region while keeping the task feasible for the 6-DoF PiPER arm.

Pick-and-place tasks. The pick-and-place group includes placing a cube into a marked region, placing a cube on top of a drawer platform, and placing a small object into a bowl. These tasks are relatively tolerant to small pose errors, so the original action chunk often remains usable. VLA-Corrector mainly helps when occasional drift occurs during approach or release.

Alignment tasks. The alignment group increases the precision requirement. In corner placement, the robot must place a cube at a specific corner of the drawer top. In square-hole insertion, the cube must be aligned with a square opening before insertion. In edge alignment, the robot must align an object with a narrow visual marker or edge. These tasks are more sensitive to accumulated error, making stale action execution more harmful near the final contact.

Disturbance recovery tasks. The disturbance group directly evaluates the open-loop blind spot. We introduce manual perturbations at predefined task phases rather than at fixed time steps. For moving-object grasp, the target object is shifted when the gripper is close to grasping it. For moving-placement target, the robot first grasps the object normally, and the target platform or placement region is shifted before release. For moving-insertion target, the square hole or alignment target is shifted when the robot is about to align or insert. The disturbance is applied once per trial, kept within the camera view, and constrained to a physically recoverable range. The same timing rule and disturbance range are used for both the baseline and VLA-Corrector.

E.4 Observed Real-World Failure Modes

Despite the improvements, VLA-Corrector still fails in several real-world cases. If the target object or fixture is moved beyond the reachable region, or if the disturbance happens after the gripper has entered an unfavorable pose, the robot may no longer have enough workspace or time to recover through a single corrective replan. In tight alignment tasks, a 6-DoF arm without force feedback may fail due to contact geometry, friction, or small height errors even when the visual target is corrected. We also observe failures caused by visual ambiguity, such as partial occlusion by the gripper or poor contrast between the object and the target region. Finally, OGG still depends on the frozen $\pi_{0.5}$ action prior: it can bias the next generation toward a better corrective direction, but it cannot create a recovery behavior that the backbone policy cannot represent.

E.5 Real-World Demonstration Examples

We provide representative real-world demonstration examples mainly for the disturbance recovery tasks. In these cases, the object or target fixture is manually shifted during execution, allowing us to visualize the practical robustness of VLA-Corrector under human-induced disturbances.

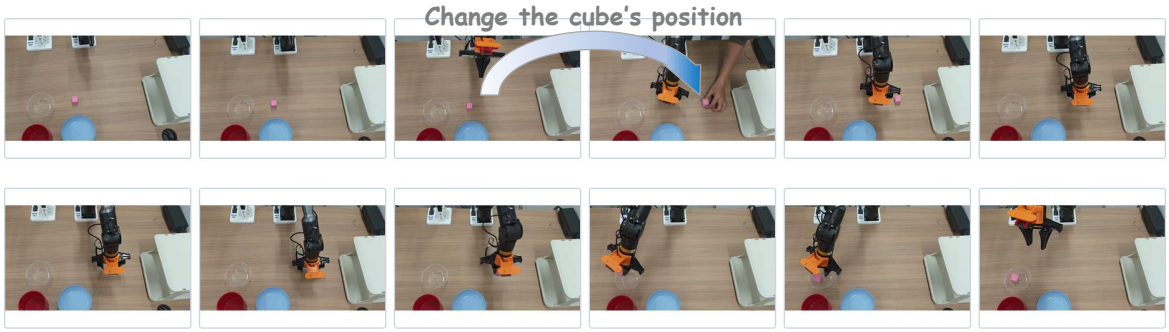


Figure 10 Demo: Moving-object grasp. The robot picks up the cube and places it into the white bowl, while a human manually changes the cube's position during the process.

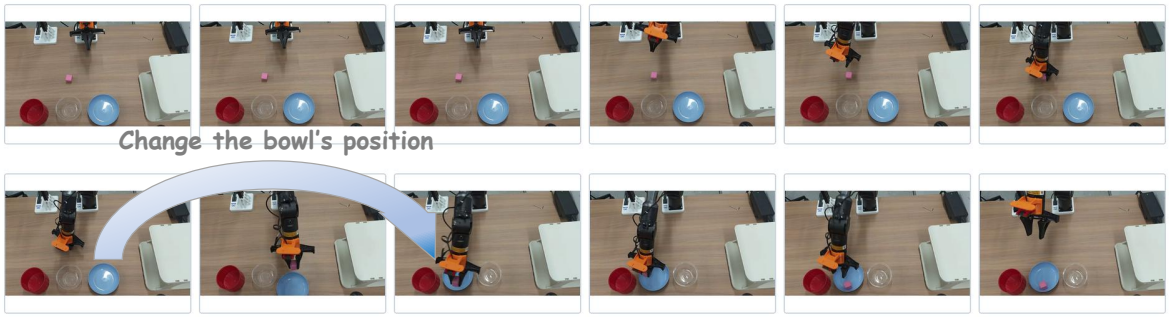


Figure 11 Demo: Moving-placement target. The robot picks up the cube and places it into the blue bowl, while a human manually changes the bowl's position during the process.

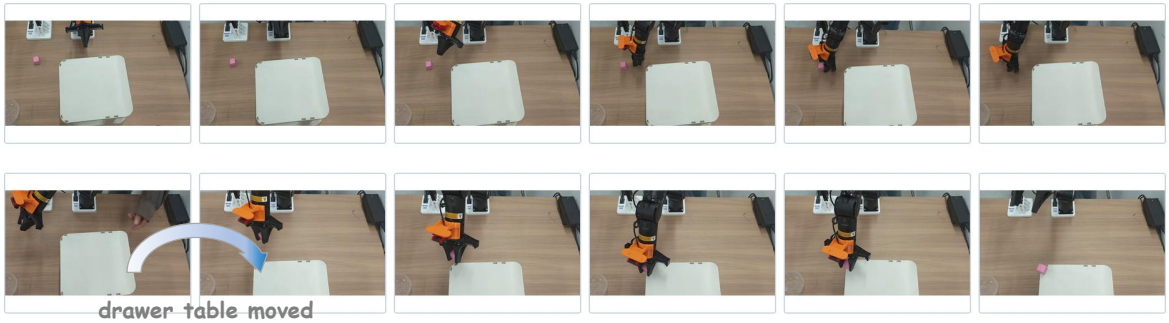


Figure 12 Demo: Moving-insertion target. The robot picks up the cube and places it at the upper-left corner of the drawer top, while a human manually changes the drawer's position during the process.